TAMBAN TWO-MODEL SYSTEM

Repository Investigation and Next-Step Improvement Plan

Model 1 fish detection + Model 2 Head/Body/Tail quality grading

Audit basis: Repository snapshot from Thesis-main.zip (GitHub archive commit f31520d5272cf9efb41072bfb35835c05a315dc5) plus the latest real-system observations: Model 1 detects fish well but produces non-fish false positives; Model 2 frequently fails to detect parts; Ungraded dominates; camera overlays and grade counts need refinement.

Prepared as a practical implementation roadmap for development, testing, thesis documentation, and defense.

1. Executive Summary

The current project is an integrated prototype rather than a finished grading system. Model 1 is already useful for whole-fish localization and counting, while Model 2 and the decision pipeline are the immediate bottlenecks. The next stage should focus on isolating Model 2 failure, correcting the active grading rules, improving negative-data coverage for Model 1, and making the interface expose the system state clearly.

Best immediate approach: Do not retrain both models at once. First prove whether Model 2 itself is weak or whether the Model 1 crop handoff is causing the failure. Run direct Model 2 inference on exact training images, compare full-frame versus crop inference, log raw part detections before grading, and only retrain after that diagnosis.

Area	Current state	Next action
Model 1 fish detection	Strong on real fish, including darker scenes; false positives on people, candles, dark objects	Hard-negative mining + threshold validation; keep current model as baseline
Model 2 parts	Critical weakness: parts rarely appear, may flicker or disappear, and even a training image reportedly failed	Direct raw-inference sanity tests; compare full-frame vs crop; verify checkpoint/threshold/padding
Grading engine	Body is required, but missing Head/Tail are currently renormalized; several YAML safeguards are not loaded	Implement original 50/30/20 weights without renormalization; add coverage, region-count and margin gates
Ungraded	Dominant in real use	Do not remove it; reduce unnecessary Ungraded by fixing Model 2 and temporal evidence
UI overlays	Live overlay omits the word Ungraded when quality is None; upload label mixes quality text with Model 1 confidence	Show Fish ID + final grade/Ungraded + separate D and Q confidence
Counts	Live quality counters are completed line-crossing events, not the distribution of all currently visible fish	Show two count groups: visible/active grades and session/crossing grades
Repository	Functional but contains duplicated datasets, generated outputs, a legacy parallel package and incomplete metadata	Clean structure after P0 runtime fixes; do not delete evidence until archived

2. What the Current Code Actually Does

The runtime behavior is important because some of the observed problems are caused by the active implementation rather than the conceptual design.

1. **Model 1 runs on the full frame.** The default confidence is 0.50 and the detector is single-class Fish. It then uses ByteTrack when available, otherwise an IoU tracker fallback. Evidence: src/api/app.py:29-38 and src/inference/yolo_fish_detector.py:270-304.
2. **Model 2 runs on a Model 1 crop, not on the original full frame.** In live mode each tracked fish is cropped and passed to Model 2. In upload mode the same crop path is used. Evidence: src/api/camera.py:299-318 and src/api/image_analysis.py:144-158.

3. **Model 2 is not run on every live frame.** The default quality interval is 3 frames. A track can therefore move or disappear between Model 2 calls. Evidence: src/api/app.py:37-38 and src/api/camera.py:303-309.
4. **The live grade waits for temporal evidence.** The grading engine defaults to two observations before a final live verdict. This is appropriate for stability but can produce Ungraded for short-lived tracks. Evidence: src/inference/grading_engine.py:58-60, 309-316.
5. **The current grading engine does not require all three body parts.** It requires Body by default. Head and Tail can be absent. This is different from the assumption that a fish must show complete Head+Body+Tail before grading.
6. **Missing-part weights are currently renormalized.** If only some parts are available, their weights are rescaled to total 1.0. Evidence: src/inference/grading_engine.py:270-279 and README.md:272-275. This conflicts with the newer desired rule that missing evidence must not be renormalized.

Important correction to the current assumption: The high Ungraded rate is not because the code requires all Head+Body+Tail. The active code mainly requires Body plus enough score and, in live mode, enough observations. Therefore the dominant Ungraded output points more strongly to weak/unstable Model 2 Body detection, crop distribution shift, or temporal timing.

3. Findings From the Real System Test

Observed behavior	Most likely technical meaning	Priority
Model 1 counts real fish correctly, even in dark areas	Strong sensitivity/recall on the intended object domain	Keep
People, candles, shadows or dark objects sometimes become Fish	The detector lacks enough hard-negative examples and/or its operating threshold is too permissive for deployment scenes	P1
Model 2 rarely displays Head/Body/Tail	The model may be weak, but the crop handoff, zero ROI padding, threshold or checkpoint must be isolated first	P0
An exact training image did not yield parts in the app	Strong diagnostic warning: a pipeline or model problem exists before generalization can be judged	P0
Model 2 detections disappear quickly	Frame-to-frame confidence instability + quality_interval=3 + no robust part persistence can cause flicker	P0/P1
Ungraded dominates	Expected downstream effect when Body or sufficient evidence is not retained	P0
Live camera does not visibly say Ungraded on the fish box	Current live overlay only adds a grade when quality is truthy	P1
Counts appear to show Class C but not Ungraded	Live dashboard counters are session crossing events; visible fish and completed-session counts are different concepts	P1

4. P0: Diagnose Model 2 Before Retraining

This is the highest-value next step. The goal is to answer one question: is Model 2 itself failing, or is the integrated crop pipeline causing a model that otherwise works to fail?

Test	Input path	What to record	Interpretation
A. Golden full-frame	Exact training image -> Model 2 directly	Raw boxes/classes/confidences at conf 0.01, 0.05, 0.10, 0.15, 0.25	If this fails badly, inspect checkpoint/training first
B. App crop	Same exact image -> Model 1 -> crop -> Model 2	Crop pixels, crop size, boxes, part classes, confidences	If A works but B fails, the handoff is the problem
C. Crop + padding	Same image with 5-15% scale-aware padding around Model 1 ROI	Whether Head/Tail recover and confidence rises	If improved, tight cropping is clipping/context-starving Model 2
D. Full-frame integrated	Same full image -> Model 1 + Model 2 independently -> association	Part recall and ownership to each parent Fish	If strong, make this the preferred architecture
E. New unseen frame	Final candidate pipeline on truly new fish	Per-fish grade, part coverage, false parts, Ungraded reason	Only use after A-D are understood

Do not lower the threshold blindly: A conf=0.01 run is a diagnostic only. It tells you whether the model produces any evidence at all. The deployment threshold must later be selected on validation data to balance missed parts against false parts.

4.1 Add a Golden Training-Image Sanity Test

Keep 5-10 known, correctly annotated training images as diagnostic fixtures. They are not evaluation data; they are a pipeline smoke test. Every runtime change should prove that the expected part classes can still be emitted from these images.

```
Golden image test record
Image: <exact training filename>
Expected regions: Head / Body / Tail
Expected grade family: <A/B/C/Rejected>
Direct Model 2 full-frame: PASS / FAIL
Model 1 crop -> Model 2: PASS / FAIL
Crop + padding: PASS / FAIL
Lowest useful confidence: ____
Notes: ____
```

4.2 Verify the Exact Checkpoint in the Running App

- At startup print Model 1 path, SHA-256, class names and input size.
- At startup print Model 2 path, SHA-256, all 12 class names and input size.
- Abort startup if Model 2 does not expose the exact trained 12-class mapping.
- Display the model hash or short build ID in the Settings/Diagnostics panel so stale checkpoints are obvious.

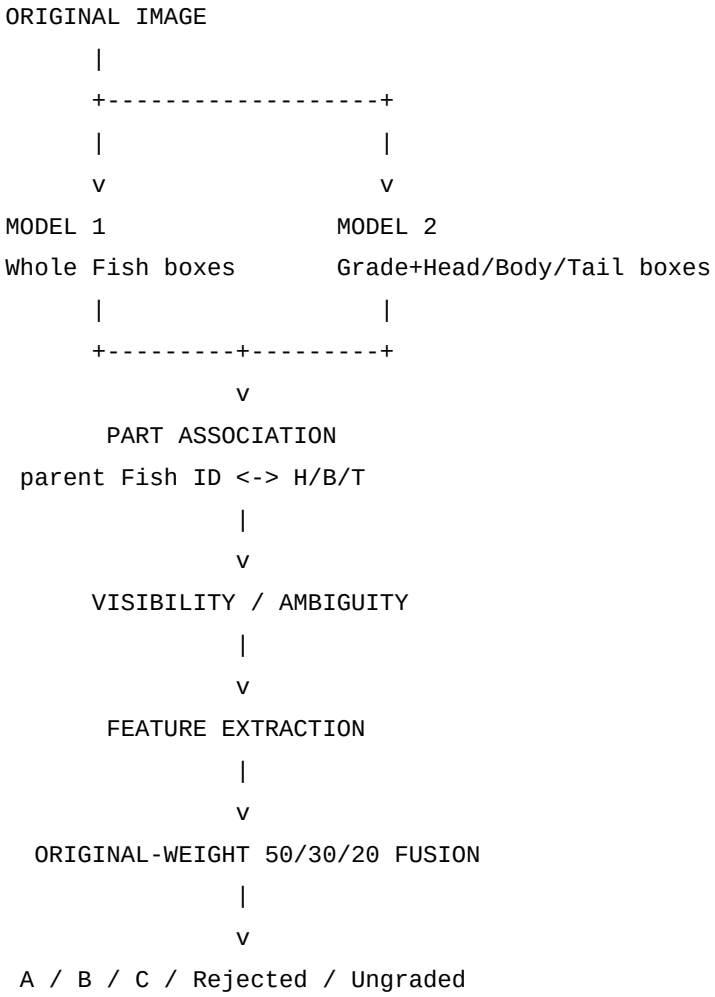
- Keep a models/registry.yaml file containing model role, filename, hash, training run, image size and class mapping. Do not commit large private checkpoints if that is not desired.

4.3 Temporarily Remove Temporal Variables During Debugging

For diagnosis only, set the live quality interval to 1 so Model 2 runs every frame that the fish is visible, and inspect raw part detections before the grading engine. After the detector is stable, restore an interval chosen from measured latency and tracking performance.

5. Recommended Architecture: Parallel Detection + Association

The strongest next architecture for this project is to compare the current crop pipeline against a parallel full-frame pipeline. Model 1 and Model 2 remain separate models, but both receive the original image. A dedicated association layer assigns each Head/Body/Tail detection to one Model 1 parent fish.



Why this is worth testing: Model 2 was trained on full-scene imagery. The current production path gives it a tight Model 1 crop. If direct full-frame inference works better, the full-frame association design removes a train-versus-inference context mismatch while preserving per-fish identity.

5.1 Association Rules

- **Parent containment.** Prefer parts whose center or substantial area lies inside a Model 1 fish box.

- **Longitudinal geometry.** Use the parent fish axis and expected Head-Body-Tail ordering as supporting evidence.
- **One slot per region.** Do not silently assign two Heads to one fish unless explicitly treated as ambiguity.
- **Global matching.** When two fish overlap, compare candidate assignments jointly instead of greedily choosing the nearest part.
- **UNASSIGNED state.** If ownership is not reliable, leave the part unassigned. Do not force a wrong parent.
- **Do not infer physical absence from nondetection.** Missing detector evidence can mean occlusion, out-of-frame, low confidence, or actual missing anatomy.

5.2 Keep Crop Mode as an Experimental Baseline

Do not delete the crop path. Add a configuration such as FISH_QUALITY_INPUT_MODE=crop|full_frame. Run both modes on the same validation images and compare part recall, false parts, association accuracy, final grading coverage and inference time. The better validated mode becomes the production default.

6. Correct the 50/30/20 Decision Logic

The current code renormalizes weights when a part is missing. That is inconsistent with the desired conservative evidence model. The next implementation should retain the original weights so missing evidence remains missing rather than being redistributed.

Example	Current renormalized behavior	Recommended behavior
Body A=0.82; Head/Tail unavailable	Body becomes 100% of available weight -> A score can remain 0.82	Body contribution = 0.82 x 0.50 = 0.41; coverage=0.50; keep Ungraded
Body B=0.92; Head B=0.84; Tail unavailable	Body/Head weights renormalize from 0.80 to 1.00	B score = 0.92x0.50 + 0.84x0.30 = 0.712; coverage=0.80; may grade if validation gates pass
Body C=0.90; Tail C=0.80; Head unavailable	Body/Tail weights renormalize from 0.70 to 1.00	C score = 0.90x0.50 + 0.80x0.20 = 0.61; coverage=0.70; may grade if validation gates pass

6.1 Recommended Decision Fields

```
weighted_score[g] = 0.50*C_body,g + 0.30*C_head,g + 0.20*C_tail,g

coverage = 0.50*I(body reliable) + 0.30*I(head reliable) + 0.20*I(tail reliable)

margin = top_grade_score - second_grade_score
```

The formulas above use only reliable, correctly associated evidence. Missing parts contribute zero and the remaining weights are not renormalized.

6.2 Provisional Starting Rules - Validate Before Thesis Claims

These are engineering starting points, not scientifically validated thresholds: Use them to build the next version, then tune them on an independent validation set. Do not present them as biological truth.

- Body must be reliably detected for an A/B/C verdict.

- At least two reliable anatomical regions should be present for the first implementation.
- A provisional coverage floor of 0.70 allows Body+Tail (0.70), Body+Head (0.80), or all three (1.00), while Body-only (0.50) remains Ungraded.
- Use a provisional top-score threshold around 0.50 only as a starting point; tune with validation.
- Add a top-versus-second margin gate so near-ties become Ungraded rather than forced A/B/C.
- Rejected must come from positive Rejected evidence or a separately validated structural rule. Nondetection, overlap or out-of-frame status alone must not mean Rejected.

6.3 Fix the Configuration Mismatch

configs/grading_engine.yaml currently contains final_verdict_threshold, minimum_regions_observed, minimum_original_weight_coverage and strict-mode values, but GradingConfig does not define those fields, so from_mapping silently ignores them. This creates a dangerous gap between what the configuration file appears to control and what the runtime actually uses.

[ ] Add final_verdict_threshold to GradingConfig and use it in final eligibility.

[ ] Add minimum_regions_observed and enforce it.

[ ] Add minimum_original_weight_coverage and enforce it.

[ ] Add a minimum_grade_margin field and enforce it.

[ ] Remove normalized_weight from the decision calculation, or keep it only as a diagnostic field clearly marked not used for final verdict.

[ ] Update README so it no longer documents renormalization after the code is changed.

7. Improve Model 1 Without Losing Its Strength

Model 1 is already the strongest component. The aim is to reduce non-fish false positives without sacrificing its ability to detect real fish in dark or difficult regions.

7.1 Hard-Negative Mining

Create a focused negative set from the exact environment where false detections happen. For YOLO detection, these can be valid images with no Fish annotations. They teach the model that the scene contains background, not an object class that must be boxed.

Negative category	Examples to capture	Reason
People/hands	person near table, hand reaching in, arm crossing camera	Observed false positives
Light sources	candle, flashlight reflection, bright hotspot	Observed false positive and shape/brightness confusion
Dark clutter	dark cloth, shadow, cables, tools, tray edges	Observed dark-area false positives
Fish-like objects	elongated tools, plastic strips, leaves, packaging	Hard negatives that resemble fish shape
Empty scenes	empty tray/background under normal and dim light	Teaches clean background rejection

- Start with roughly 100-300 carefully chosen hard-negative images rather than thousands of random internet images.
- Include the exact objects that triggered false positives during testing.
- Keep a separate non-fish validation set so you can measure false positives per frame after fine-tuning.

- Sweep Model 1 confidence thresholds after fine-tuning; do not simply raise 0.50 until false positives disappear because that can also remove dark real fish.

7.2 Optional Second-Stage Fishness Check

Only if hard-negative fine-tuning is insufficient, test a lightweight second-stage binary verifier on each Model 1 candidate crop: Fish versus Not Fish. This is optional because it adds latency and complexity. Prefer improving Model 1 training data first.

8. Improve Model 2 After the Diagnostic Tests

Retraining trigger: Retrain Model 2 only if direct full-frame inference on known training images and correctly prepared validation images is still poor after confirming the checkpoint, class mapping, threshold and image size.

8.1 Data Priorities

- Collect more Class A and Class B part examples because the existing class counts are much lower than Class C.
- Prioritize B-versus-C mixed scenes under the same lighting and background. This reduces scene/session shortcuts.
- Collect overlapping and partially occluded fish deliberately, but label only visible anatomical regions correctly.
- Keep the same physical fish within one split. Do not let repeated photographs of one specimen appear in train and test.
- Fill specimen identity metadata. The current dataset/metadata.csv and dataset/manifests/specimen_groups.csv contain headers but no specimen records in the audited snapshot.
- Avoid treating every augmentation as an independent specimen. Augmented copies must stay with their original source split.

Current Model 2 class	Count in repository research artifact
Grade_A_Body	275
Grade_A_Head	230
Grade_A_Tail	226
Grade_B_Body	470
Grade_B_Head	477
Grade_B_Tail	288
Grade_C_Body	3,042
Grade_C_Head	2,343
Grade_C_Tail	1,379
Rejected_Body	1,208

Current Model 2 class	Count in repository research artifact
Rejected_Head	1,126
Rejected_Tail	556

The imbalance does not prove that imbalance alone caused the failure, but it is large enough that per-class recall must be inspected. A single overall mAP can hide very weak A/B/Tail behavior.

8.2 If Crop Mode Wins

- Train/fine-tune Model 2 on crops that match the exact runtime crop policy.
- Use scale-aware ROI padding so Head/Tail are not cut off by Model 1 box error.
- Preserve aspect ratio; avoid unnecessary stretch distortion.
- Include full, partial, rotated and overlapping fish crops so inference conditions match deployment.

8.3 If Full-Frame Mode Wins

- Keep Model 2 on original frames.
- Activate the association utilities already present in src/inference/part_fusion.py as the starting point.
- Evaluate association separately from detection so a correct part box assigned to the wrong fish is counted as an association error, not a detection error.
- Keep UNASSIGNED as a legitimate outcome when ownership is ambiguous.

9. Reduce Flicker Without Hiding Detector Problems

Temporal smoothing should make a good detector stable; it should not be used to disguise a detector that rarely emits correct parts.

1. **During debugging, run Model 2 every frame.** Set the quality interval to 1 and log raw detections.
2. **After detection is reliable, add short persistence.** A part observed reliably in recent frames can remain available briefly when one frame misses it.
3. **Use track-aware evidence.** Only retain part history while the parent Fish ID is stable.
4. **Expire stale evidence.** Do not carry a Head/Body/Tail detection indefinitely after the fish moves, overlaps or changes orientation.
5. **Separate visibility from quality.** Use states such as PRESENT, NOT_DETECTED, OCCLUDED, OUT_OF_FRAME, AMBIGUOUS, POSSIBLY_MISSING, UNKNOWN. These are not grades.

10. UI Changes Recommended From the Test

UI issue	Current code behavior	Recommended change
No Ungraded label on live fish box	src/api/camera.py adds quality text only when quality is truthy	When grades are enabled, always draw quality or "Ungraded"
Confidence meaning is unclear	Upload label shows quality text followed by Model 1 detector confidence	Use explicit D=detector and Q=quality labels; never imply D confidence is grade confidence
Part labels are hard to see	Live part text is tied to the features overlay toggle	Add a dedicated Parts overlay toggle; show Region + Grade + part confidence

UI issue	Current code behavior	Recommended change
Counts seem incomplete	Live grade counters are completed inspection-line events	Show Current Visible and Session Counted as separate panels
No reason for Ungraded on box	Side analysis contains explanations but camera overlay does not	Show compact reason code: NO_BODY, LOW_COVERAGE, AMBIGUOUS, WAITING_TEMPORAL

10.1 Recommended Camera Label

```
Fish #03 | Class C | Q 78% | D 94%
Fish #04 | UNGRADED | D 91% | LOW_COVERAGE
Fish #05 | REJECTED | Q 86% | D 96%
```

Q is Model 2/fusion evidence. D is Model 1 detection confidence. They are different quantities and should never be presented as one combined calibrated probability.

10.2 Counts: Current Visible vs Session Counted

```
CURRENTLY VISIBLE
Total 6 | A 1 | B 0 | C 1 | Rejected 0 | Ungraded 4

SESSION COUNTED ACROSS LINE
Total 19 | A 5 | B 3 | C 7 | Rejected 1 | Ungraded 3
```

The repository already keeps an Ungraded bucket in SessionStatistics. The confusing part is semantics: those quality counters increment when a tracked fish crosses the virtual inspection line. They are not a distribution of every fish currently visible in the camera. Add a separate active-quality distribution from active_tracks.

Required invariant: For both upload results and any visible-fish grade summary: A + B + C + Rejected + Ungraded must equal the corresponding total fish count.

11. Repository Structure and Cleanup

Do cleanup after the runtime P0 items are stabilized so working evidence is not accidentally deleted. The audited snapshot contains valuable research artifacts but also a large amount of duplication and parallel legacy code.

Finding	Evidence from audited snapshot	Action
Duplicate dataset export	Root Dried Fish Quality Grading.v7i.coco-segmentation and dataset/Dried Fish Quality Grading.v7i.coco-segmentation contain the same 915 files byte-for-byte	Keep one canonical copy; archive/remove the duplicate after verifying references
Image redundancy	4,224 JPG files but only 1,488 unique JPG hashes; 2,736 redundant JPG copies	Do not keep every export/copy in main. Keep provenance manifests and one canonical dataset representation
Legacy parallel package	fish_quality_system/ is largely self-contained and not imported by the main src/ runtime	Move to legacy/ or archive branch if no longer used

Finding	Evidence from audited snapshot	Action
Generated logs in root	dashboard.stdout.log, dashboard.stderr.log, run_status.json, multiple research plots/CSVs	Move reproducible outputs under results/ and ignore transient logs/status
Model layout incomplete	Documented models/fish_detector/weights and models/fish_quality/weights checkpoints are absent from the repository snapshot	Add model registry + setup instructions + hashes; keep weights outside Git if desired
Empty training modules	src/training/Yolov26_First.py and Yolov26_Second.py are zero-byte files	Remove or replace with actual entry points
Incomplete framework modules	src/evaluation/evaluate.py and src/preprocessing/preprocess.py contain NotImplementedError	Finish or clearly mark/archive so users do not treat them as working
Specimen metadata empty	metadata.csv and specimen_groups.csv have headers only	Populate before claiming specimen-independent validation

11.1 Recommended Repository Shape

```
Thesis/
  README.md
  requirements.txt
  configs/
    runtime.yaml
    grading_engine.yaml
    model_registry.yaml
  src/
    api/
    inference/
    association/
    features/
    evaluation/
    preprocessing/
    training/
  frontend/
  tests/
  data/
    manifests/
    metadata/
    README.md
  models/
    README.md
    registry.yaml
  results/
    baselines/
```

```
    validation/
    figures/
  docs/
    methodology/
    experiments/
    legacy/
  legacy/
    fish_quality_system/
```

12. Automated Test Status and Technical Debt

The repository test suite was executed on the audited snapshot. Result: 194 passed, 4 failed, 3 skipped, with additional subtests passing. The failures should be cleared before the next thesis milestone so regression checks are trustworthy.

Failing test	What it indicates	Recommended handling
test_replay_uses_recorded_part_evidence_and_rules_without_retraining	Expected weighted score differs by 0.005 from implementation	Review expected math while changing fusion; replace renormalized expectation with the finalized original-weight specification
test_segmentation_interval_reuses_grade_then_applies_late_correction	Test mock refers to FishSegmentation without defining/importing it	Fix the test fixture first; then keep this regression test for late grade correction
test_diagonal_major_axis_is_rotated_horizontal	Orientation normalization did not make the diagonal major axis horizontal	Fix before relying on geometry/width features for B/C
test_specimen_manifest_rejects_duplicate_instance_link	SpecimenAssociation now requires annotation_index but loader does not supply it	Fix schema/loader contract before specimen-safe split validation

Runtime impact: Not every failing test explains the live Model 2 failure. The Model 2 diagnosis remains P0. However, the orientation and specimen-manifest failures directly affect future geometry research and trustworthy evaluation, so they should not be left unresolved.

13. Evaluation Plan After the Fixes

Component	Metric / evidence	Why it matters
Model 1 fish detector	Precision, recall, mAP50, mAP50-95, count MAE, false positives per non-fish frame	Measures fish sensitivity and the candle/person problem
Model 2 raw parts	Per-class precision/recall/F1/AP for all 12 labels; region recall for Head/Body/Tail	Reveals whether A/B/Tail classes are being missed
Model 2 sanity	Detection success on golden training images	Pipeline smoke test only; not generalization evidence
Full-frame vs crop	Part recall, false parts, latency, final grade coverage on the same validation set	Determines the best handoff architecture scientifically

Component	Metric / evidence	Why it matters
Association	Correct parent assignment rate, unassigned rate, ambiguous rate	Separates ownership errors from detector errors
Final grading	Per-class precision/recall/F1, confusion matrix, coverage, Ungraded rate	Measures useful end-to-end performance
Safety/abstention	Error rate among graded fish + rate/reasons of Ungraded	Checks whether abstention prevents forced wrong grades
UI/counting	A+B+C+Rejected+Ungraded=Total; active versus session counts	Prevents misleading operator displays

13.1 Experimental Sequence

1. Freeze one repository revision and record model hashes.
2. Run Model 2 golden-image sanity test.
3. Run full-frame versus crop A/B experiment.
4. Select one architecture on validation data only.
5. Fix grading engine to non-renormalized 50/30/20 and implement coverage/margin gates.
6. Tune thresholds using validation data.
7. Fine-tune Model 1 with hard negatives and re-evaluate real-fish recall.
8. Add temporal smoothing only after raw Model 2 is stable.
9. Fix UI overlays and dual count semantics.
10. Run end-to-end validation on independent specimens/scenes.
11. Lock weights, thresholds, association and fusion policy.
12. Run untouched final test set once for thesis results.

14. Practical Development Roadmap

Phase	Goal	Deliverable / exit condition
P0 - Pipeline truth	Find why Model 2 misses even known images	Direct/full-frame/crop comparison table; verified checkpoint; raw detection logs
P0 - Fusion correctness	Make code match intended evidence policy	No renormalization; coverage and margin fields active; tests updated
P1 - Model 1 robustness	Reduce non-fish false positives	Hard-negative set + before/after false-positive benchmark
P1 - UI clarity	Make every visible fish understandable	Fish ID + grade/Ungraded on box; D/Q separate; active and session counts
P1/P2 - Model 2 recovery	Improve raw part recall if diagnosis shows model weakness	Targeted retraining/fine-tuning with independent validation
P2 - Association	Handle overlap without mixing fish	Validated parent-part association + UNASSIGNED
P2 - Temporal stability	Reduce flicker without stale evidence	Validated short-window smoothing

Phase	Goal	Deliverable / exit condition
P3 - Thesis evaluation	Produce defensible final results	Locked system + untouched test metrics + failure analysis

15. Master Checkbox Checklist

Use this section as the working development checklist. Complete P0 items before major retraining or feature expansion.

P0 - Model 2 diagnosis

[ ] Confirm the exact Model 2 checkpoint path and SHA-256 used by the app.
[ ] Confirm all 12 Model 2 class names at startup.
[ ] Run exact training image directly through Model 2 at conf=0.01, 0.05, 0.10, 0.15, 0.25.
[ ] Save raw part predictions before grading.
[ ] Run the same image through Model 1 crop -> Model 2.
[ ] Test crop padding at several scale-aware values.
[ ] Test Model 2 on the original full frame.
[ ] Compare full-frame and crop part recall on at least 20 labeled images.
[ ] Set quality_interval=1 during diagnosis.
[ ] Decide whether the bottleneck is checkpoint/model, crop handoff, threshold, or temporal logic.

P0 - Grading engine correctness

[ ] Remove decision-weight renormalization for missing parts.
[ ] Calculate original evidence coverage.
[ ] Enforce minimum_regions_observed.
[ ] Enforce minimum_original_weight_coverage.
[ ] Load and enforce final_verdict_threshold from YAML.
[ ] Add a top-versus-second grade margin threshold.
[ ] Keep Body as required for A/B/C until validation supports another policy.
[ ] Keep Body-only cases Ungraded in the first implementation.
[ ] Keep missing/nondetected/occluded separate from Rejected.
[ ] Update unit tests and README to match final fusion behavior.

P1 - Model 1 false-positive reduction

[ ] Capture false-positive examples from people, hands, candles, shadows and non-fish objects.
[ ] Create 100-300 hard-negative training images with no Fish boxes.
[ ] Create a separate non-fish validation benchmark.
[ ] Fine-tune Model 1 without changing the original baseline checkpoint.
[ ] Sweep confidence threshold after fine-tuning.
[ ] Compare fish recall and false positives before/after.

P1 - UI and counts

[ ] Draw "Ungraded" on live fish boxes when no final grade exists.
[ ] Draw Fish ID and final grade directly inside/on the camera overlay.

[ ] Label Model 1 confidence as D and grade/fusion confidence as Q.
[ ] Add a dedicated part-overlay toggle.
[ ] Show a compact Ungraded reason.
[ ] Add current visible A/B/C/Rejected/Ungraded counts.
[ ] Keep session crossing counts as a separate panel.
[ ] Assert A+B+C+Rejected+Ungraded equals the corresponding total.
[ ] Add automated tests for upload and live count invariants.

P1/P2 - Model 2 data and training

[ ] Populate physical specimen IDs and capture-session metadata.
[ ] Collect additional A and B examples, especially tails.
[ ] Collect mixed B+C scenes under the same lighting/background.
[ ] Collect overlap/occlusion cases intentionally.
[ ] Keep physical specimens isolated across train/validation/test.
[ ] Choose training data format that matches the selected runtime input mode.
[ ] Retrain/fine-tune only after P0 diagnosis.
[ ] Report per-class and per-region performance, not only overall mAP.

P2 - Association and stability

[ ] Enable a full-frame Model 2 experimental path.
[ ] Associate parts to parent Fish IDs using geometry and global matching.
[ ] Allow UNASSIGNED when ownership is ambiguous.
[ ] Add visibility states separate from grade.
[ ] Add short-window part persistence after raw detection is stable.
[ ] Expire stale evidence when track/geometry changes.
[ ] Evaluate overlap scenes separately.

P2 - Repository cleanup

[ ] Archive or remove the duplicate 915-file v7 COCO export after verifying references.
[ ] Reduce redundant image copies; keep one canonical representation plus manifests.
[ ] Move fish_quality_system/ to legacy/ if it is not part of the active product path.
[ ] Remove/replace zero-byte training modules.
[ ] Finish or archive NotImplementedError entry points.
[ ] Move root logs and generated research outputs under results/.
[ ] Add models/registry.yaml with hashes and provenance.
[ ] Keep .gitignore aligned with generated artifacts and local checkpoints.

P3 - Validation and thesis evidence

[ ] Fix all automated test failures.
[ ] Create independent validation and untouched test sets using specimen/session grouping.
[ ] Tune thresholds on validation only.
[ ] Lock model hashes and decision policy before final test.
[ ] Report Model 1 detection + false-positive metrics.
[ ] Report Model 2 per-class/region metrics.
[ ] Report association accuracy and UNASSIGNED rate.

[ ] Report final A/B/C/Rejected metrics and Ungraded rate.
[ ] Include representative success and failure cases in the thesis.
[ ] Document limitations honestly, especially B/C similarity, overlap, lighting and dataset provenance.

16. Acceptance Gates Before Calling the System "Improved"

Gate	Pass condition
Pipeline sanity	Known training images produce expected raw Model 2 part evidence under direct inference; model hash and class map are confirmed.
Architecture selection	Full-frame versus crop comparison has been run on the same labeled validation set; the selected mode is evidence-based.
Fusion correctness	Missing evidence is not renormalized; coverage/region/margin gates are active and tested.
Model 1 robustness	False positives on a dedicated non-fish benchmark are measured and reduced without unacceptable loss of real-fish recall.
UI clarity	Every visible parent fish shows ID + grade or Ungraded; detector and quality confidence are distinct.
Counting integrity	Visible distribution and session-crossing distribution are separated; each distribution sums correctly.
Repository health	Automated tests pass; metadata schema is consistent; active vs legacy code is clear.
Thesis readiness	Independent specimen/session-safe validation and untouched final test are completed after policy lock.

17. What Not to Do Yet

- Do not remove Ungraded just to make the dashboard look more decisive.
- Do not retrain Model 1 from scratch when its real-fish detection is already strong.
- Do not lower Model 2 confidence to 0.01 in production; use it only for diagnostics.
- Do not treat a missing Head/Tail detection as physical damage or Rejected.
- Do not combine Model 1 confidence and Model 2 grading evidence into one probability.
- Do not tune on the final test set.
- Do not delete duplicate/legacy folders until they are archived and references are verified.
- Do not claim the all-data training-pool metrics are independent final accuracy.

18. Final Recommended Order of Work

1. **Model 2 raw sanity first.** Direct full-frame inference on exact training images and raw confidence sweep.
2. **Compare full-frame versus crop.** This decides whether the integration architecture is the main problem.
3. **Fix fusion semantics.** No renormalization; enforce evidence coverage and ambiguity gates.

4. **Fix live timing.** Run every frame during debugging, then add controlled smoothing.
5. **Improve Model 1 with hard negatives.** Keep the existing strong detector as baseline.
6. **Fix UI and count semantics.** Visible fish and session-crossing counts should never be confused.
7. **Only then retrain Model 2 if needed.** Use better specimen-safe, balanced, mixed-scene data.
8. **Validate, lock, test.** Once the pipeline is stable, freeze the policy and run independent final evaluation.

Bottom line: Your project does not need a complete restart. Model 1 is already a useful foundation. The highest-return work is to make Model 2 observable and reliable, align its inference input with its training distribution, correct the evidence-fusion implementation, then validate the integrated system with independent fish.

Appendix A. Key Code Evidence From the Audited Snapshot

Finding	Repository reference
Model 1 default confidence 0.50	src/api/app.py:29-32
Model 2 default confidence 0.25	src/api/app.py:33-38
Model 2 default image size 640	src/api/app.py:36
Model 2 quality interval defaults to 3	src/api/app.py:37
Live path crops each Fish before Model 2	src/api/camera.py:299-318
Upload path crops each Fish before Model 2	src/api/image_analysis.py:144-158
Live overlay only adds grade when quality is truthy	src/api/camera.py:175-186
Upload overlay label uses Model 1 detector confidence	src/api/image_analysis.py:166-170
Current decision weights are renormalized over available parts	src/inference/grading_engine.py:270-279
Body is required for final grade by default	src/inference/grading_engine.py:56 and 300-315
Live verdict waits for minimum track observations	src/inference/grading_engine.py:58-60 and 309-315
Quality counters contain Ungraded bucket	src/api/domain.py:18 and 247-283
Quality counters increment on completed crossing events	src/api/domain.py:407-432
Several YAML safeguards are not GradingConfig fields	configs/grading_engine.yaml vs src/inference/grading_engine.py:49-81

Appendix B. Audit Notes

Repository-level numeric observations from this snapshot: 4,224 JPG files, 1,488 unique JPG hashes, and 2,736 redundant JPG copies. The two v7 COCO-export locations contain the same 915 files byte-for-byte. dataset/metadata.csv and dataset/manifests/specimen_groups.csv contain schema headers without specimen records. The active documented Model 1 and Model 2 default checkpoint directories are not included in this archive, while an older exp-4 part-preview checkpoint is present. These facts affect repository size, reproducibility and evaluation provenance but do not by themselves prove a model-quality failure.

Automated tests executed for this audit: 194 passed, 4 failed, 3 skipped. The report above distinguishes runtime-critical Model 2 debugging from test failures that primarily affect evaluation, geometry or metadata workflows.